Assignment 2

Question 1

Since all numbers are distinct , without loss of generality we can assume any n consecutive numbers .
Example:
1 2 3 4 5
1 3 2 4 5
1 3 2 5 4
So we have to swap the maximum of $A[i-1], A[i], A[i+1]$ with $A[i]$, where i is an odd index .

Algorithm

```
Iterate through the array
Check if the index is odd
Swap it with the maximum of A[i-1] , A[i] , A[i+1]
```

In CPP

```
void swapVariables(int &a , int &b) {
    int temp = a ;
    a = b ;
    b = temp;
}
void peakValley(int* a , int n) {
    for(int i = 1 ;i <= n ; i+=2) {
        int curr = i ;
        if(a[i-1] > a[curr]) {
            curr = i - 1 ;
        }
        if(i+1 <= n && a[i+1] > a[curr]) {
            curr = i + 1;
        }
        if(curr != i) {
            swapVariables(a[i] , a[curr]) ;
        }
    }
}
```

Proof Of Correctness

By looking at the given example , I can say that if position of $A[i]$ is changed the peak so formed will not be affected by any other iteration as we do `i+=2` .
Also we are checking at every iteration for the maximum of three consecutive elements skipping the previously formed peak.

Time and Space Complexity

The loop runs for $n/2$ times ,
Each iteration does

- constant comparisons
- atmost 1 swap

Thus $T(n) = \Theta(n)$

We don't use any extra space for array , only few variables like `curr , i , temp` , thus space complexity will be $O(n)$.

Question 2

Algortihm

Using two pointer method , take 2 indices :

- `left = 0`
- `right = n-1`

Get $sum = A[left] + A[right]$

There are 3 cases which are possible :

- $sum = x$, print the pair then skip duplicates from both sides
- $sum < x$, increase left
- $sum > x$, decrease right

The algorithm is $\Theta(n)$ as both we move through the array only once because `left` will move through `0 to n-1` and `right` will move through `n-1 to 0` .
Each element is visited at most once therefore :

> $T(n) = \Theta(n)$

C++ Implementation

```
void getPairs(int* a , int n , int x) {
    int left = 0 ;
    int right = n-1 ;

    while(left < right) {
        int sum = a[left] + a[right] ;

        if(sum == x) {
            cout<<a[left]<<" "<<a[right]<<endl;
            int u = a[left] ;
            int v = a[right] ;

            while(right >= 0 && a[right] == v) {
                right--;
            }
            while(left < n && a[left] == u) {
                left++;
            }
        }
        else if(sum > x) {
            right--;
        }
        else{
            left++;
        }
    }
}
```

Maximum possible cases

Assuming all elements to be distinct and consecutive to get the maximum possible case.
This makes the situation in which we use every element once and get the same sum for for all the distinct pairs .

Example: `{1,2,3,4,5,6,7}` and `x = 7`
There will be 3 pairs .

Hence , we can write maximum number of pairs as

$$\left\lfloor \frac{n}{2} \right\rfloor$$

Question 3

Algorithm

Traverse the array once.
For each element:

- If it is `0` , swap it with the element at `pos_0`
- Increment `pos_0`

Minimum number of adjacent swaps

For 1st part we only needed to swap latest `0` and the first occured `1` ,
But now we can only do adjacent swaps so the number of swaps will increase each time, we would have to move all occured ones to right of newly occuring `0` .

Get 2 variables :

- `cnt_1` -> number of ones occured till now
- `swaps` -> total swaps needed

Traverse through array

- If element is `1` : add it to `cnt_1`
- If element is `0` : add `cnt_1` to `swaps`

Example : `A = [1,0,1,0,0]`

Element	cnt_1	swaps added
1	1	0
0	1	+1
1	2	0
0	2	+2
0	2	+2

Total swaps = 5

C++ Implementation

```
void sortBin(int* a , int n) {
    int pos_0 = 0;
    for(int i = 0 ; i < n ; i++) {
        if(a[i] == 0) {
            int temp = a[pos_0] ;
            a[pos_0] = a[i] ;
            a[i] = temp ;
            pos_0++;
        }
    }
}


long long int minSwaps(int* a , int n) {
    long long int swaps = 0 ;
    int cnt_1 = 0 ;
    for(int i = 0 ; i < n ; i++) {
        if (a[i]== 1){
            cnt_1++ ;
        }
        else {
            swaps += cnt_1 ;
        }
    }
    return swaps ;
}
```

Time & Space Complexity

Both algortihm are of $\Theta(n)$ Time complexity and $O(n)$ Space complexity.
We traverse through array only once in both cases , and use only few variables like
`pos_0 , cnt_1 , temp , swaps`

Question 4

Algorithm

Can solve this problem by Reversal algorithm.

Split array in to parts reverse those two parts and then reverse the whole array to get the desired resultant array .

$$BA = (A^R B^R)^R$$

where R means Reverse .

Steps

- Reverse the first k elements
- Reverse the last $n - k$ elements
- Reverse the entire array

C++ Implementation

```
void reverseArray(int* a , int n) {
    for(int i = 0 ; i < n/2 ; i++) {
        int temp = a[i] ;
        a[i]  = a[n - i - 1] ;
        a[n - i - 1] = temp;
    }
}
void leftRotate(int* a , int n , int k){
    reverseArray(a , k) ;
    reverseArray((a + k) , n - k) ;
    reverseArray(a , n) ;
}
```

Time and Space Complexity

Each reversal takes $O(length)$.

Operation	Time
Reverse first k	O(k)
Reverse last n-k	O(n-k)
Reverse whole array	O(n)

Total: $O(n)$

We use `temp` variable as extra space thus Space complexity is $O(1)$.

Question 5

a)

To solve this problem, we can desgin our algorithm as:

1. First we sort the array.
2. We recursively start to traverse the array at each level and only put unique values at the current place (quite easy to do as the duplicates are grouped together)
3. We do this until we fill our auxiliary array upto r values after which we just print them and call return.
4. Due to the array being traversed recursively, we can remove the last added value easily after which we call the recursive function itself for the next value.

b)

```
#include <vector>
#include <algorithm>
using namespace std;

void printCombination(const vector<int>& current) {
    for (int x : current) {
        cout << x << " ";
    }
    cout << "\n";
}

void findCombinations(vector<int>& a, int r, int start, vector<int>& current) {
    if (current.size() == r) {
        printCombination(current);
        return;
    }

    for (int i = start; i < a.size(); i++) {
        if (i > start && a[i] == a[i - 1]) continue;

        current.push_back(a[i]);
        findCombinations(a, r, i + 1, current);
        current.pop_back();
    }
}
```

c)

The time complexity of this algorithm would be: $O(r \cdot N)$, where N represents the number of possible combinations which in case of all distinct elements becomes ${}^{n}C_{r}$ making the absolute worst case time complexity: $O(r \cdot {}^{n}C_{r})$

d)

The space complexity includes:

- The storing of sorted array, giving us an $O(n)$ space complexity.
- The recursion stack which goes to a maximum depth of r giving us an $O(r)$ space complexity.
- The storing of the possible combinations, which gives us an $O(r \cdot {}^{n}C_{r})$ space complexity

Therefore the overall space complexity is: $O(r \cdot {}^{n}C_{r})$

Question 6

A number has to satisfy 2 conditions

- It must be **Centered Heptagonal Number**
- It must be **prime**

For checking 1st condition

$$x = \frac{7n^2 - 7n + 2}{2}$$

n gets simplified to:

$$n = \frac{7 + \sqrt{56x - 7}}{14}$$

x is Centered Heptagonal Number iff

1. $7 + \sqrt{56x - 7}$ divisible by 14
2. $56x - 7$ is perfect square.

```
#include<cmath>

bool isPrime(long long int x) {
    if(x<2) return false ;

    for(int i = 2 ; i*i <= x ; i++) {
        if(x%i==0) return false ;
    }
    return true ;
}

bool isCenteredHeptagonal(long long int x) {
    long long int y = 56*x - 7 ;
    long long int z = sqrt(y) ;
    if(y!=z*z) return false ;

    if((7+z)%14!=0) return false ;

    return true ;
}

int countCHprimes(int* a, int n) {
    int count = 0 ;
    for(int i = 0 ; i < n ; i++) {
        if(isPrime(a[i]) && isCenteredHeptagonal(a[i])) {
            count++;
        }
    }
    return count ;
}
```

Time & Space Complexity

- n is array size
- M is maximum value in array

Prime check takes upto $\sqrt{x}$

> $O(\sqrt{M})$

Heptagonal Check does only square root , multiplication , modulus

> $O(1)$

Total complexity $O(n\sqrt{M})$

Space complexity would be $O(1)$ as we have stored only few variables like `count , y , z` .

Question 7

By observing Move() function :

- Total sum of array remains constant.

Assuming

- Every even indice to be `x`
- Odd indice to be `x-1`
- Number of even indices `num_even`
- Number of odd indices `num_odd`
- Sum of initial array elements `sum`

Number of elements `n` in array is odd.

Also, as all the even indices are 1 greater than odd indices , this makes us reach the condition :

```
n*x = sum + num_odd
```

- sum + num_odd is divisible by n .

Minimum cost calculation

$$k = \frac{\sum |A[i] - target[i]|}{2}$$

where $target[i]$ = `x` for even indices and `x-1` for odd indices.

```
#include<iostream>
#include<vector>
#include <cmath>
using namespace std;

bool isZigZag(int* a ,int n) {
    if(n%2==0) return false ;

    int num_odd = n/2 ;
    int num_even = n - num_odd ;

    long long int sum = 0 ;
    for(int i = 0 ;i < n; i++) {
        sum+=a[i];
    }
    if((sum+num_odd)%n!=0) return false ;

    return true ;
}
long long int minCost(int* a ,int n) {
    if(!isZigZag) return -1 ;
    long long int sum = 0 ;
    for(int i = 0 ;i < n; i++) {
        sum+=a[i];
    }
    long long cost = 0 ;
    int num_odd = n/2 ;
    int num_even = n - num_odd ;

    long long x = (sum + num_odd) / n ;
    for(int i = 0; i < n; i++) {
        if(i % 2 == 0) {
            cost += abs(a[i] - x);
        } else {
            cost += abs(a[i] - (x - 1));
        }
    }

    return cost / 2 ;
}
```

Time & Space COmplexity

It will take $O(n)$ time as we traeverse through the array only once whether doing the sum or finding the total cost .

Space complexity will be $O(1)$ as we have only few variables like `num_odd` , `num_even` , `etc.`

Question 8

a)

To solve this problem, we aim to transform string s into the target string using the minimum number of operations. Since we can only replace characters (implied by the fixed length of the target comparison), we iterate through the strings and whenever $s[i]$ does not match $target[i]$, we update $s[i]$.

b)

```
#include <iostream>
#include <string>

int main() {
    string s, target;
    getline(std::cin, s);
    getline(std::cin, target);

    int n = s.length();
    int m = target.length();

    for (int i = 0; i < m && i < n; i++) {
        if (s[i] != target[i]) {
            s[i] = target[i];
        }
    }

    cout << "Modified string: " << s << "\n";
    return 0;
}
```

c)

The time complexity is $\Theta(N)$, where N is the length of the string, as we perform a single pass. The space complexity is $O(1)$ auxiliary space (excluding the storage for strings).

d)

If the strings are of different lengths and the operation allowed includes deletion or insertion, the problem would shift toward Edit Distance (Levenshtein Distance), requiring a Dynamic Programming approach with $O(n \cdot m)$ complexity.

e)

For very large strings, this linear approach is optimal as it ensures each character is visited exactly once, making it suitable for real-time stream processing of character differences.

Question 9

```
sum = 0;
for ( int i = 1; i <= n; i++) {
    for ( int j = 1; j <= n; j *= 4){
        sum += 1;
    }
}
int *arr ;
arr = new int [sum];
```

- For Outer loop
  - It runs `n` time i.e. $O(n)$
- For Inner loop
  - It runs for $\log_4 n$ times i.e. $O(\log n)$

After last iteration $sum = n \cdot \log_4 n$

Space Complexity

Array of size `sum` is allocated .

> Space Complexity $\Theta(n \log n)$
> Other variables `(i, j, sum, pointer)` use constant space.

Question 10

A *word* is defined as a sequence of non-space characters separated by one or more spaces.

Algorithm

We traverse the string only once and count a word **only when**:

- The current character is *not a space* , and
- It is either:
  - The first character of the string, or
  - The previous character is a space.

Algorithm Steps

1. Initialize `count = 0`
2. Traverse the string from index `0` to `n-1`
3. For each character:
   - If current character is not a space **and**
   - (i == 0 OR previous character is a space)
     - Increment `count`
4. Return `count`

Time and Space Complexity

- Time Complexity:$O(n)$
  (Single traversal if the string)
- Space Complexity: $O(1)$
  (Only a counter variable is used)

C++ Implementation

```
#include <string>

int countWords(const string &str) {
    int count = 0;
    int n = str.length();

    for (int i = 0; i < n; i++) {
        if (str[i] != ' ' && (i == 0 || str[i - 1] == ' ')) {
            count++;
        }
    }

    return count;
}
```

Question 11

```
int recursiveSum( int n) {
    if (n <= 0) {
        return 0;
    }
    int partialSum = 0;
    for ( int i = 1; i <= n; i++) {
        partialSum += recursiveSum( i / 2);
    }
    return partialSum + n;
}
```

Time Complexity Analysis

For any given n the loop runs from 1 to n , in each iteration we call `recursiveSum(i/2)`
Thus, the time complexity satisfies :

$$T(n) = \sum_{i=1}^{n} T(i/2) + \Theta(n)$$

After simplifying

$$T(n) \approx 2\sum_{k=1}^{n/2} T(k) + \Theta(n)$$

For finding the upper bound , each time $T(n)$ calls n new recursions , even if the input size is halved ($i/2$) .
This structure suggests an exponential growth rate because the "branching factor"
Thus, the complexity is exponential

$$T(n) = O(2^n)$$

Space Complexity
Space complexity is determined by recursion depth. Since the input size is halved in every recursive step (e.g., $n \to n/2 \to n/4 \ldots$), the maximum depth of the recursion tree is:

$$S(n) = O(\log n)$$

Question 12

We solve this problem by backtracking

- To find all derangements , we maintain a used boolean array and a condition:

```
if (current_index == original_index_of_val) continue;
```

C++ Code

```cpp
#include <vector>
using namespace std ;

void findDerangements(vector<int>& original, vector<int>& current, vector<bool>& used) {
    int n = original.size();
    if (current.size() == n) {
        for (int x : current) cout << x << " ";
        cout << "\n";
        return;
    }

    int i = current.size();
    for (int j = 0; j < n; j++) {
        if (!used[j] && original[j] != original[i]) {
            used[j] = true;
            current.push_back(original[j]);
            findDerangements(original, current, used);
            current.pop_back();
            used[j] = false;
        }
    }
}
```

Time & Space Complexity

The time complexity is $O(n \cdot d_n)$, where d_n is the number of derangements. Since $d_n \approx n!/e$, the complexity is effectively $O(n \cdot n!)$.

The space complexity for the algorithm consists of 2 parts:

- Storing the derangements will require an extra space of $O\left(n \cdot n!\right)$.
- The recursion stack will occupy an extra $O\left(n\right)$ space for finding the permutations.

$\Rightarrow$ The overall space complexity is $O\left(n \cdot n!\right)$.

Question 13

a)

Using recursive backtracking approach to find all combinations of r elements from the n given elements, but we have to print those combinations where the sum of the elements is exactly equal to m

b)

The time complexity is $O(r \cdot \binom{n}{r})$ in the worst case. While the m constraint allows to discard branches early, we still potentially explore all combinations.

c)

The time complexity does not depend on m in the naive-solution, however as m increases in size, the DP approach makes significant savings in time.

d)

Yes, the number of valid combinations is always $\leq \binom{n}{r}$. Adding a sum constraint m can only reduce the number of valid subsets from the total possible subsets of size r.

e)

If $m = \sum a_i$ and $r = n$, there is only 1 possible combination. If m is very large or very small (outside the range of possible sums), the number of valid combinations is 0.

Question 14

a)

To generate all possible permutations given k fixed positions, we treat the fixed indices as constants. we only permute the set of values not used in the fixed positions over the remaining $(n-k)$ indices.

Time Complexity

The time complexity is $O(n \cdot (n-k)!)$ because we are generating $(n-k)!$ permutations, and each takes $O(n)$ time to construct and print.

c)

As k increases, $(n-k)$ decreases, significantly reducing the number of permutations. If $k = n$, the complexity is $O(n)$ as there is only one configuration.

d)

The total number of permutations is $(n-k)!$, assuming all remaining elements are distinct.

e)

There is only 1 permutation if $k = n$ or $k = n - 1$. There are 0 permutations if the fixed values provided are not part of the original set or if there are duplicate assignments to the same index.

Question 15

a)

The sum of the main diagonal and anti-diagonal can be found in $O(n)$ time. We iterate from $i = 0$ to $n - 1$ and add $M[i][i]$ and $M[i][n - i - 1]$. If n is odd, the middle element $M[n/2][n/2]$ is counted twice, so we subtract it once.

b)

It can be found in a single loop of n iterations by simply plugging the formula present in $a)$ part in the code.

```
int diagonalSum(int** mat, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += mat[i][i];
        sum += mat[i][n - i - 1];
    }
    if (n % 2 != 0) {
        sum -= mat[n / 2][n / 2];
    }
    return sum;
}
```

c)

The time complexity is $O(n)$ because we only use a single loop traversing the dimension of the matrix, not n^2 elements.

d)

For a sparse matrix stored as a list of triplets $(row, col, value)$, we simply iterate through the list and add the value if $row == col$ or $row + col == n - 1$. This is $O(E)$ where E is the number of

non-zero elements.

e)

If the matrix is mirrored, the indices for the diagonals may swap, but since we are summing both, the total S remains invariant.

Question 16

a)

To find the sum of all unique prime factors of all elements in an array, we use a Sieve of Eratosthenes to precompute the smallest prime factor (SPF) for all numbers up to the maximum element M. Then, for each element, we factorize it using the SPF and store unique factors in a set.

b)

The time complexity is $O(M \log \log M + n \log M)$. The sieve takes $O(M \log \log M)$ and factorizing each of the n elements takes $O(\log M)$ using the SPF array.

c)

True. Since 1 is not a prime, the smallest prime is 2. Any prime factor $p \geq 2$. Thus, the sum of prime factors will always be greater than the count of distinct prime factors.

d)

The simple division method is better if the array contains very few, very large primes (where M is huge but n is small). The sieve is better for many smaller numbers.

e)

If $A = [1, 2, \ldots, n]$, the unique prime factors are all primes $\leq n$. The sum would be the sum of all primes up to n, which is approximately $O(\frac{n^2}{\log n})$ by the Prime Number Theorem.

Question 17

a)

```
long long F(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return F(n - 1) + F(n - 2) + n * n;
}
```

b)

The closed form of the recursive relation can be found by calculating its CF and PF.

Its CF is simply the linear combination of expponential form of the roots of the quadratic formed by the recursive terms:

$$CF = a\left(\frac{1+\sqrt{5}}{2}\right)^n + b\left(\frac{1-\sqrt{5}}{2}\right)^n$$

Whereas, the PF is found to be:

$$PF = -n^2 - 6n - 13$$

Now the actual $F(n)$ is $CF + PF$.
So putting the values $F(0) = 0$ and $F(1) = 1$, we get the values of a and b as,

$$a = \frac{13}{2} + \frac{29\sqrt{5}}{2},\ b = \frac{13}{2} - \frac{29\sqrt{5}}{2}$$

Which actually gets quite assimilated into the exponents to get the salient form of:

$$F(n) = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{n+7} - \left(\frac{1-\sqrt{5}}{2}\right)^{n+7}\right) - n^2 - 6n - 13$$

c)

The time complexity of the Recursive Algorithm is simply the recursive function without polynomials $(Fibbonaci\ Series)$:

$$T(n) = T(n-1) + T(n-2)$$

with $T(1) = 1,\ T(2) = 1$, which has the exponential solution:

$$T(n) = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right)$$

d)

We can calculate $F(n)$ instantaneously using the closed form, but there is always a possibilty of floating-point inaccuracies. We can also calculate it in $O(n)$ time using a simple for loop or we could use the matrix method to calculate $F(n)$ in $O(\log n)$ time.

e)

As $n \to \infty$, $F(n) \to \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^{n+7}$ as the other exponent term tends to 0 and the polynomic terms are very weak in comparison.

Question 18

a)

The determinant of N (where $N_{ij} = A_{ij}/R_i$ and R_i is the row sum) is $det(N) = \frac{det(A)}{\prod_{i=1}^{n} R_i}$. We calculate $det(A)$ and divide by the product of all row sums.

b)

If we consider an $n \times n$ matrix A then the above steps will take:

- $O(n^2)$ for the row sum calculation.
- $O(n^2)$ for the calculation of determinant of A using Arnoldi Iteration.
- $O(n)$ for dividing the row sums from $det(A)$. This could also be done in $O(1)$ if we simply calculate a cumulative product of the row sums but that risks overflow and also does not affects the overall tiime complexity by a big margin, so it can be ignored.

Therefore the time complexity of the algorithm is $O(n^2)$.

c)

An interesting mathematical property of N would be that at least one of its eigen values will always be 1 provided that N is defined as the sum of all the rows is 1.

d)

$det(N)$ can be greater than 1. If $A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$, then $R_i = 10$, and $N = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, so $det(N) = 1$. But if $A = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$, $R_i = 1$, so $det(N) = det(A) = 3 > 1$.

e)

If A is singular, $det(A) = 0$, which implies $det(N) = 0$.

Question 19

a)

The derivative $P'(x)$ of $P(x) = \sum a_i x^i$ is $\sum i \cdot a_i x^{i-1}$.

$$P'(x) = na_n x^{n-1} + (n-1)a_{n-1}x^{n-2} + \cdots + 2a_2 x + a_1$$

b)

In C++, we represent the polynomial as an array coeff[] where coeff[i] is the coefficient of x^i. The derivative array `der[]` will have `der[i] = coeff[i+1] * (i+1)` .

c)

The time complexity to find the coefficients of the derivative is $O(n)$.

d)

To find the k-th derivative, the coefficient of x^i becomes $a_{i+k} \cdot \frac{(i+k)!}{i!}$.

e)

The interesting properties for large k is that if it becomes greater than n, then we simply make the resulting polynomial 0.

Question 20

a)

To move n disks from A to C via B such that no disk moves directly from A to C:

1. Move $n-1$ disks from A to C.
2. Move disk n from A to B.
3. Move $n-1$ disks from C to A.
4. Move disk n from B to C.
5. Move $n-1$ disks from A to C.

b)

The recurrence is $T(n) = 3T(n-1) + 2$. With $T(1) = 2$, the solution is $T(n) = 3^n - 1$.

c)

The time complexity is $O(3^n)$, which is much slower than the standard Tower of Hanoi ($2^n - 1$).

d)

Yes, this is the minimum number of moves under the "no direct $A \to C$" constraint.

e)

The number of moves triples for every additional disk because each disk must visit the intermediate peg B twice.

Question 21

a)

For any integer n, we can calculate $P(n)$ in $O(k)$ time. Assuming that N is the last value which gives a prime, and the primality test takes at most $O(N^{\frac{k}{2}})$ time.

b)

Up until about 10^{12}, the division method is completely fine. However for much larger numbers, we would have to use Milan's Primality test which would reduce our time complexity.

c)

As discussed earlier, using the simple division method, we will get the time complexity of:

$$O(N \cdot k + N^{\frac{k}{2}+1}) \approx O(N^{\frac{k}{2}+1})$$

However, considering the Milan's Primality test approach, we could reduce our time complexity to about:

$$O(N \cdot (k + k^3 \log^3 N)) \approx O(N \cdot k^3 \log^3 N)$$

d)

Given the constraint, it can be proven that the pattern will always break before $n = a_0 \Rightarrow N < a_0$, as $P(a_0)$ will be guaranteed to be a multiple of a_0. This helps us to set an upper bound to the loop but it overall doesn't change anything else.

Question 22

a)

To minimize $F(B)$, we analyze the term $e^{B_{ij}}$. Since $e^{x+iy} = e^x(\cos y + i \sin y)$, to keep the result real and minimal, y should be π (making the term $-e^x$).

b)

If $y_{ij} = \pi$ for one element and 0 for others, $F(B)$ becomes negative. We set one $B_{ij} = S + i\pi$ and others to 0.

c)

The complexity to construct the matrix is $O(n^2)$.

d)

The optimal matrix B changes with respect to the value of S just as the algorithm discusses:

- $S \geq \pi$: The matrix is a complex sparse matrix with any one entry equal to $\left(\sqrt{S^2 - \pi^2} + i\pi\right)$.
- $S < \pi \text{ and } S < \frac{2n^2 \ln n}{n^2 - 1}$: The matrix is a real matrix with S distributed equally among all entries making them equal to $\frac{S}{n^2}$.
- $S < \pi \text{ and } S \geq \frac{2n^2 \ln n}{n^2 - 1}$: The matrix is a complex sparse matrix with any one entry equal to S.

Submitted by

- Garv

2511035